

FITSYNC DOCUMENTATION

FITSYNC: AI-Powered Digital Twin for Real-Time Heart Rate Monitoring

1. Introduction

Fitsync is an innovative AI-driven system designed to enhance workout safety by providing real-time heart rate monitoring and anomaly detection. It addresses the increasing number of cardiac-related incidents during physical exercise by offering a **Digital Twin-based monitoring system** that continuously tracks heart rates, detects anomalies, and provides predictive insights using **machine learning (ML) models**.

This system combines **MATLAB Simulink, Arduino-based BPM sensors, an LSTM Autoencoder for anomaly detection, and a web-based interface** to ensure users can monitor their heart rate trends effectively. The primary goal of Fitsync is to **alert users of potential health risks, enabling them to take proactive measures** before a critical situation arises.

2. Problem Statement

Recent reports have highlighted a growing number of sudden cardiac-related deaths during workouts. Traditional gyms and fitness trainers can offer dietary and exercise advice, but they **lack real-time physiological monitoring capabilities**. Even wearables like smartwatches may not provide **historical trend analysis or anomaly detection** tailored to an individual's past data.

Fitsync is designed to fill this gap by offering:

- ✓ **Real-time heart rate tracking** using BPM sensors.
 - ✓ **Anomaly detection** using **ML-based pattern recognition**.
 - ✓ **Digital Twin-based simulation** for personalized analysis.
 - ✓ **Web dashboard visualization** for user interaction and alerts.
-

3. Fitsync Architecture & Methodology

3.1 Core Components

Fitsync consists of **four major components**:

1. **Hardware Input** – Capturing real-time heart rate data using an **Arduino Uno + BPM Sensor**.
 2. **Machine Learning (ML) Model** – An **LSTM Autoencoder** trained on fitness data to detect heart rate anomalies.
 3. **Digital Twin Implementation (MATLAB Simulink)** – A simulation environment that models, processes, and predicts heart rates.
 4. **Web Dashboard (Node.js, MongoDB, UDP Communication)** – A **live visualization tool** that displays heart rate trends and alerts users.
-

3.2 ML Model for Anomaly Detection

3.2.1 Preprocessing Heart Rate Data

- The dataset consists of user heart rate data over time, stored in CSV format.
- **Data Normalization:** Min-Max Scaling applied to scale heart rates between 0 and 1.
- **Sequence Generation:** Each user's data is split into **50-time-step sequences** for LSTM training.

3.2.2 LSTM Autoencoder Architecture

- **Encoder:** Uses an **LSTM** to capture sequential patterns in heart rate fluctuations.
- **Latent Representation:** A bottleneck **latent space** reduces dimensionality.
- **Decoder:** Another **LSTM** reconstructs the input heart rate sequence.
- **Anomaly Detection:** The model **compares reconstructed vs. actual heart rates**. A large reconstruction error indicates an anomaly.

3.2.3 Setting the Threshold for Anomalies

The anomaly threshold is determined based on **Mean Squared Error (MSE) distribution** from the training data. If an input sequence deviates beyond a statistically defined threshold, it is flagged as an anomaly. This threshold can be adjusted dynamically based on **user-specific trends**.

How does this help?

- **Personalized Anomaly Detection:** Each user's normal heart rate pattern is learned.
 - **Risk Warnings:** If heart rates fluctuate abnormally, **alerts can be sent to the user in real-time**.
-

3.3 Digital Twin Implementation in MATLAB Simulink

3.3.1 System Structure

The **Digital Twin model** consists of:

1. **Live Heart Rate Input** → Data streamed from Arduino BPM sensor to MATLAB Simulink.
2. **Preprocessing Block (reshape_data)** → Data is formatted for ML model inference.
3. **LSTM Prediction Block** → Predicts the next heart rate value.
4. **Anomaly Detection Block** → Compares real vs. predicted HR to detect anomalies.
5. **Output Blocks & UDP Communication** → Sends HR and anomaly status to the web dashboard.

Key MATLAB Simulink Blocks Used:

- **Serial Input (from Arduino)** – Reads real-time HR data.
 - **MATLAB Function Blocks** – Implements preprocessing, LSTM inference, and anomaly detection.
 - **UDP Send Blocks** – Transmits HR and anomaly results to the web dashboard.
 - **Scope & Display Blocks** – Visualizes real-time heart rate trends.
-

3.4 Web Dashboard & Real-Time Alerts

The **Node.js-based web interface** displays:

- ✓ **Live Heart Rate Graph** – Actual vs. Predicted HR trends.
- ✓ **Anomaly Detection Graph** – Highlighting abnormal heart rate patterns.
- ✓ **User Alerts** – When an anomaly is detected, a notification is triggered.

Implementation Details:

- **MongoDB** stores historical heart rate data for trend analysis.
- **UDP Communication** allows MATLAB Simulink to send real-time HR data to the web interface.
- **Alert System** – Users receive notifications when their HR **deviates significantly from normal trends**.

4. Future Enhancements

- ✓ **Personalized AI models** – Adaptive models trained per user.
- ✓ **Wearable Device Integration** – Support for smartwatches & fitness bands.
- ✓ **Integration with Medical Services** – Alerting medical professionals in case of critical anomalies.

5. Conclusion

Fitsync is a **first-of-its-kind real-time Digital Twin** implementation for fitness monitoring. By leveraging **ML-driven anomaly detection, MATLAB Simulink simulation, and a real-time web dashboard**, it empowers fitness enthusiasts with **personalized heart rate monitoring and proactive health risk assessment**.

Why is this the best model?

- **Custom-trained LSTM Autoencoder for Anomaly Detection** 
- **Live Data Streaming & Real-Time Digital Twin Simulation** 
- **User-Friendly Web Dashboard & Alerts** 
- **Preemptive Health Risk Warnings** 

With Fitsync, users can **exercise confidently, knowing their heart is in safe hands!** ❤️